Relatório da primeira análise do SonarQube

Projeto TaskQ - Back-end | Branch codex/testes-controllers | Commit 633b2b62d32b660a7bc3de13220d0c51a1f54783

Análise executada em 24/10/2026 com SonarQube Community Build 24.12.0.100206. Este relatório registra o estado inicial do projeto, antes das correções solicitadas para a segunda análise.

1. Resumo da análise

81.2%	93	0	0	43	5
Cobertura geral	Testes executados	Bugs	Vulnerabilidades	Code smells	Hotspots

A cobertura geral foi de 81.2% e está abaixo da meta de 90% definida para o trabalho. A cobertura de linhas ficou em 81.8% (57 linhas sem cobertura de um total de 314) e a cobertura de condições ficou em 77.4% (12 condições sem cobertura de 53).

O Quality Gate apareceu como APROVADO, porém a instância não retornou condições configuradas. Por isso, o resultado não substitui a meta acadêmica de 90% de cobertura. Os 93 testes terminaram sem falhas, erros ou testes ignorados.

2. Indicadores encontrados

Indicador	Resultado	Leitura
Bugs	0	Nenhum bug identificado pelas regras ativas.
Vulnerabilidades	0	Nenhuma vulnerabilidade confirmada.
Code smells	43	Pontos de manutenção e clareza que devem ser tratados antes da segunda análise.
Hotspots de segurança	5	Todos relacionados à configuração de CORS e precisam de revisão manual.
Duplicação	0.0%	Nenhum bloco de código duplicado foi medido.
Dívida técnica	128 min	Estimativa do Sonar para corrigir os code smells registrados.

3. Code smells

Foram encontrados 43 code smells: 1 blocker, 3 críticos, 9 major, 10 minor e 20 informativos. Desses, 6 estão no código principal e 37 nos testes.

Regra	Qtd.	Explicação
java:S5786	20	Modificador public desnecessário em classe ou método de teste JUnit 5.
java:S1128	5	Import não utilizado; deve ser removido para manter o arquivo limpo.
java:S1854	5	Valor atribuído a uma variável sem ser usado depois.
java:S1481	5	Variável local declarada, mas não utilizada pelo teste.
java:S6813	4	Injeção de dependência por campo; o Sonar recomenda injeção pelo construtor.
java:S1192	2	Texto repetido no código; deve ser extraído para uma constante.
java:S2699	1	Caso de teste sem nenhuma asserção.
java:S1186	1	Método vazio sem explicação ou implementação.

4. Pontos relevantes no código principal

Sev.	Arquivo / linha	Ponto encontrado
CRITICAL	src/main/java/uniamerica/tasksq_back_end/service/ProjectService.java:38	Texto repetido no código; deve ser extraído para uma constante.
CRITICAL	src/main/java/uniamerica/tasksq_back_end/service/TaskService.java:51	Texto repetido no código; deve ser extraído para uma constante.

Sev.	Arquivo / linha	Ponto encontrado
MAJOR	src/main/java/uniamerica/tasksq_back_end/controller/ProjectController.java:23	Injeção de dependência por campo; o Sonar recomenda injeção pelo construtor.
MAJOR	src/main/java/uniamerica/tasksq_back_end/controller/ProjectController.java:25	Injeção de dependência por campo; o Sonar recomenda injeção pelo construtor.
MAJOR	src/main/java/uniamerica/tasksq_back_end/controller/TaskController.java:22	Injeção de dependência por campo; o Sonar recomenda injeção pelo construtor.
MAJOR	src/main/java/uniamerica/tasksq_back_end/controller/TaskController.java:24	Injeção de dependência por campo; o Sonar recomenda injeção pelo construtor.

5. Hotspots de segurança

Os cinco hotspots são da regra java:S5122. Eles não são vulnerabilidades confirmadas: indicam que a configuração de CORS deve ser revisada para garantir que somente origens necessárias estejam liberadas.

Arquivo / linha	Situação
src/main/java/uniamerica/tasksq_back_end/controller/HolidayController.java:13	CORS habilitado - revisar origem permitida
src/main/java/uniamerica/tasksq_back_end/controller/ProjectController.java:20	CORS habilitado - revisar origem permitida
src/main/java/uniamerica/tasksq_back_end/controller/RoleController.java:18	CORS habilitado - revisar origem permitida
src/main/java/uniamerica/tasksq_back_end/controller/TaskController.java:19	CORS habilitado - revisar origem permitida
src/main/java/uniamerica/tasksq_back_end/controller/UserController.java:20	CORS habilitado - revisar origem permitida

6. Conclusão da primeira análise

O projeto não apresentou bugs, vulnerabilidades confirmadas nem duplicação de código. O principal ponto pendente é elevar a cobertura de 81,2% para pelo menos 90%. Também devem ser tratados os 43 code smells e revisados os cinco hotspots de CORS. Depois dessas correções, o SonarQube deve ser executado novamente para compor o relatório comparativo da segunda análise.

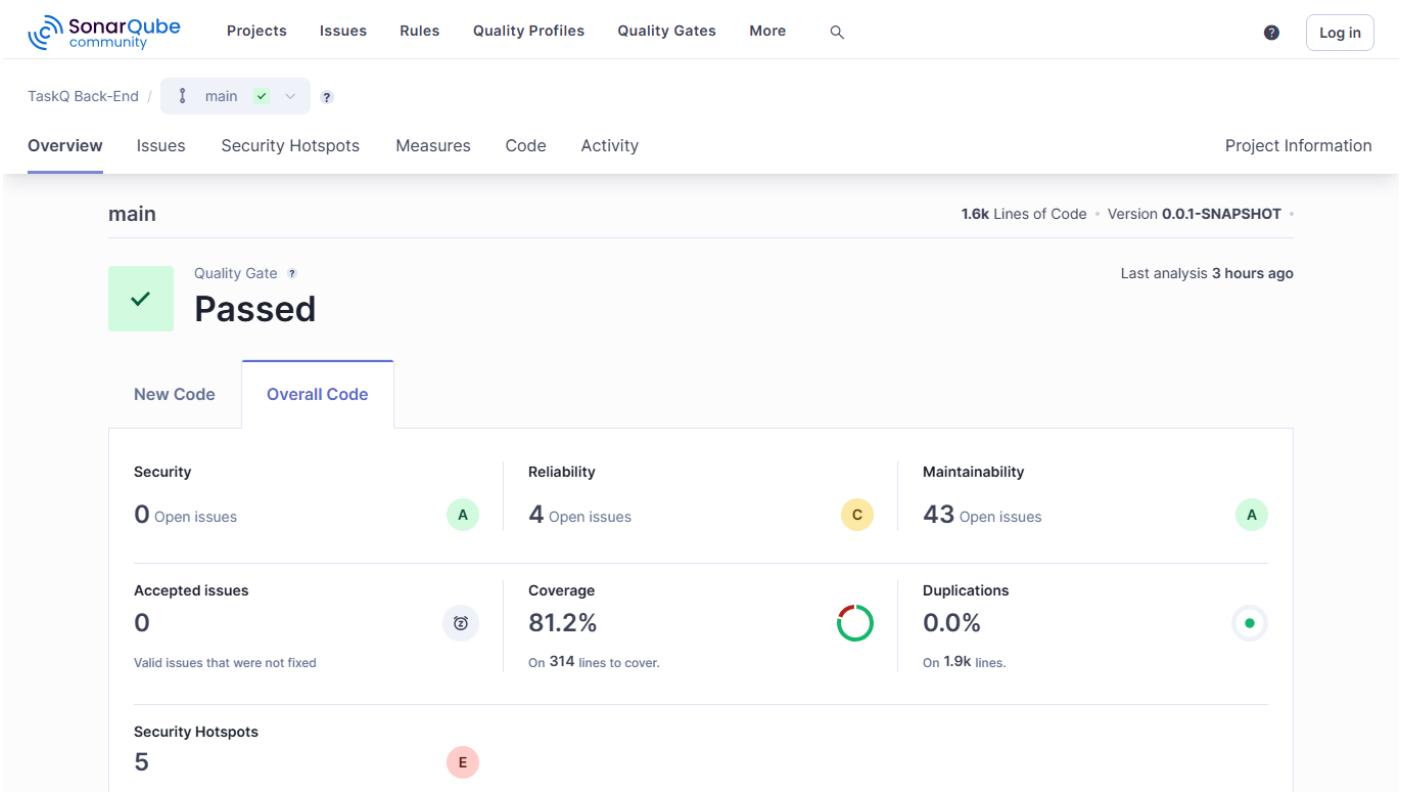
Anexo A - Lista completa dos 43 apontamentos

#	Sev.	Regra	Arquivo / linha	Mensagem
1	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/controller/UserControllerTest.java:24	Remove this 'public' modifier.
2	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/controller/RoleControllerTest.java:22	Remove this 'public' modifier.
3	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/service/RoleServiceTest.java:31	Remove this 'public' modifier.
4	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/service/XpServiceTest.java:31	Remove this 'public' modifier.
5	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/response/ProjectResponseTest.java:12	Remove this 'public' modifier.
6	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/response/RoleResponseTest.java:7	Remove this 'public' modifier.
7	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/response/TaskResponseTest.java:12	Remove this 'public' modifier.
8	MINOR	java:S1128	src/test/java/uniamerica/tasksq_back_end/dto/response/UserResponseTest.java:4	Remove this unused import 'uniamerica.tasksq_back_end.entity.enums.TaskPriority'.
9	MINOR	java:S1128	src/test/java/uniamerica/tasksq_back_end/dto/response/UserResponseTest.java:5	Remove this unused import 'uniamerica.tasksq_back_end.entity.enums.TaskStatus'.
10	MINOR	java:S1128	src/test/java/uniamerica/tasksq_back_end/dto/response/UserResponseTest.java:9	Remove this unused import 'java.time.LocalDate'.
11	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/response/UserResponseTest.java:14	Remove this 'public' modifier.
12	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/PasswordUpdateRequestTest.java:9	Remove this 'public' modifier.
13	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/ProjectRequestTest.java:11	Remove this 'public' modifier.
14	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/RoleRequestTest.java:9	Remove this 'public' modifier.
15	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/TaskRequestTest.java:13	Remove this 'public' modifier.
16	MINOR	java:S1128	src/test/java/uniamerica/tasksq_back_end/dto/request/UserRequestTest.java:8	Remove this unused import 'java.time.LocalDate'.
17	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/UserRequestTest.java:12	Remove this 'public' modifier.
18	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/dto/request/UserUpdateRequestTest.java:10	Remove this 'public' modifier.
19	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/controller/HolidayControllerTest.java:16	Remove this 'public' modifier.
20	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/controller/TaskControllerTest.java:17	Remove this 'public' modifier.
21	BLOCKER	java:S2699	src/test/java/uniamerica/tasksq_back_end/TasksqBackEndApplicationTests.java:14	Add at least one assertion to this test case.
22	MINOR	java:S1128	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:8	Remove this unused import 'org.springframework.beans.factory.annotation.Autowired'.
23	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:23	Remove this 'public' modifier.
24	MAJOR	java:S1854	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:49	Remove this useless assignment to local variable "aux".
25	MINOR	java:S1481	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:49	Remove this unused "aux" local variable.
26	MINOR	java:S1481	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:76	Remove this unused "response" local variable.
27	MAJOR	java:S1854	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:76	Remove this useless assignment to local variable "response".
28	MAJOR	java:S1854	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:81	Remove this useless assignment to local variable "aux".
29	MINOR	java:S1481	src/test/java/uniamerica/tasksq_back_end/controller/ProjectControllerTest.java:81	Remove this unused "aux" local variable.
30	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/service/HolidayServiceTest.java:20	Remove this 'public' modifier.
31	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/service/TaskServiceTest.java:28	Remove this 'public' modifier.

#	Sev.	Regra	Arquivo / linha	Mensagem
32	MINOR	java:S1481	src/test/java/uniamerica/tasksq_back_end/service/ProjectServiceTest.java:152	Remove this unused "project" local variable.
33	MAJOR	java:S1854	src/test/java/uniamerica/tasksq_back_end/service/ProjectServiceTest.java:152	Remove this useless assignment to local variable "project".
34	INFO	java:S5786	src/test/java/uniamerica/tasksq_back_end/service/ProjectServiceTest.java:25	Remove this 'public' modifier.
35	MAJOR	java:S1854	src/test/java/uniamerica/tasksq_back_end/service/ProjectServiceTest.java:76	Remove this useless assignment to local variable "resultado".
36	MINOR	java:S1481	src/test/java/uniamerica/tasksq_back_end/service/ProjectServiceTest.java:76	Remove this unused "resultado" local variable.
37	CRITICAL	java:S1192	src/main/java/uniamerica/tasksq_back_end/service/ProjectService.java:38	Define a constant instead of duplicating this literal "Projeto não encontrado" 3 times.
38	CRITICAL	java:S1192	src/main/java/uniamerica/tasksq_back_end/service/TaskService.java:51	Define a constant instead of duplicating this literal "Tarefa não encontrada" 4 times.
39	MAJOR	java:S6813	src/main/java/uniamerica/tasksq_back_end/controller/ProjectController.java:23	Remove this field injection and use constructor injection instead.
40	MAJOR	java:S6813	src/main/java/uniamerica/tasksq_back_end/controller/ProjectController.java:25	Remove this field injection and use constructor injection instead.
41	MAJOR	java:S6813	src/main/java/uniamerica/tasksq_back_end/controller/TaskController.java:22	Remove this field injection and use constructor injection instead.
42	MAJOR	java:S6813	src/main/java/uniamerica/tasksq_back_end/controller/TaskController.java:24	Remove this field injection and use constructor injection instead.
43	CRITICAL	java:S1186	src/test/java/uniamerica/tasksq_back_end/TasksqBackEndApplicationTests.java:10	Add a nested comment explaining why this method is empty, throw an UnsupportedOperationException or complete the implementation.

Evidencia visual - primeira analise do SonarQube

Captura em navegador local, com enquadramento de tela de 1366 x 768 pixels.



Quality Gate: Passed | Cobertura: 81,2% | Duplicacoes: 0,0% | Hotspots: 5

Dashboard real do projeto TaskQ Back-End executado localmente em 25/10/2026.

RELATORIO DA SEGUNDA ANALISE DO SONARQUBE

Projeto TaskQ - Back-end | Branch codex/testes-controllers | Commit c272e952167fad810a9686daed84efad1cb7ef87
Analise executada em 25/10/2026 com SonarQube Community Build 24.12.0.100206.

0	81,5%	93	5
Code smells	Cobertura geral	Testes aprovados	Hotspots revisados

Resultado: os 43 code smells da primeira analise foram corrigidos. O projeto permaneceu sem bugs, vulnerabilidades confirmadas e duplicacao. A cobertura subiu de 81,2% para 81,5%, mas ainda nao atingiu a meta academica de 90%.

1. Comparacao entre as analises

Indicador	1a analise	2a analise	Evolucao
Quality Gate	Aprovado	Aprovado	Mantido
Cobertura geral	81,2%	81,5%	+0,3 p.p.
Cobertura de linhas	81,8%	82,2%	+0,4 p.p.
Cobertura de condicoes	77,4%	77,4%	Sem alteracao
Testes	93	93	Todos aprovados
Bugs	0	0	Mantido
Vulnerabilidades	0	0	Mantido
Code smells	43	0	43 corrigidos
Divida tecnica	128 min	0 min	Eliminada
Duplicacao	0,0%	0,0%	Mantida
Hotspots	5	5	Revisados; aguardam origem do front

A cobertura de linhas passou a considerar 320 linhas, com as mesmas 57 linhas sem cobertura. As seis novas linhas vieram dos construtores adicionados aos controllers, por isso o percentual geral teve pequena variacao.

2. Correcoes realizadas

As correcoes foram pontuais e seguiram o padrao ja existente no projeto, sem incluir MockMVC, novas camadas ou alteracoes nas regras de negocio.

Regra	Qtd.	Correcao aplicada
java:S5786	20	Remocao de modificadores public desnecessarios em classes de teste JUnit 5.
java:S1128	5	Remocao de imports nao utilizados.
java:S1854	5	Remocao de atribuicoes cujo valor nao era utilizado.
java:S1481	5	Remocao de variaveis locais sem uso.
java:S6813	4	Troca de injecao por campo por construtores simples em ProjectController e TaskController.
java:S1192	2	Criacao de constantes para as mensagens repetidas de projeto e tarefa nao encontrados.
java:S2699	1	Inclusao de verificacao com assertDoesNotThrow no teste do metodo main.
java:S1186	1	Inclusao de comentario explicando o teste vazio de carregamento do contexto.

3. Validacao executada

Testes automatizados	93 executados, 0 falhas, 0 erros e 0 ignorados.
Analise SonarQube	Processada com sucesso para o commit c272e95.
Issues abertas	0 bugs, 0 vulnerabilidades e 0 code smells.
Duplicacao	0,0% nas 1,9 mil linhas analisadas.

4. Hotspots de seguranca

Os cinco hotspots continuam classificados como TO_REVIEW pela regra java:S5122. Eles apontam o uso de CORS nos controllers Holiday, Project, Role, Task e User. Nao sao vulnerabilidades confirmadas.

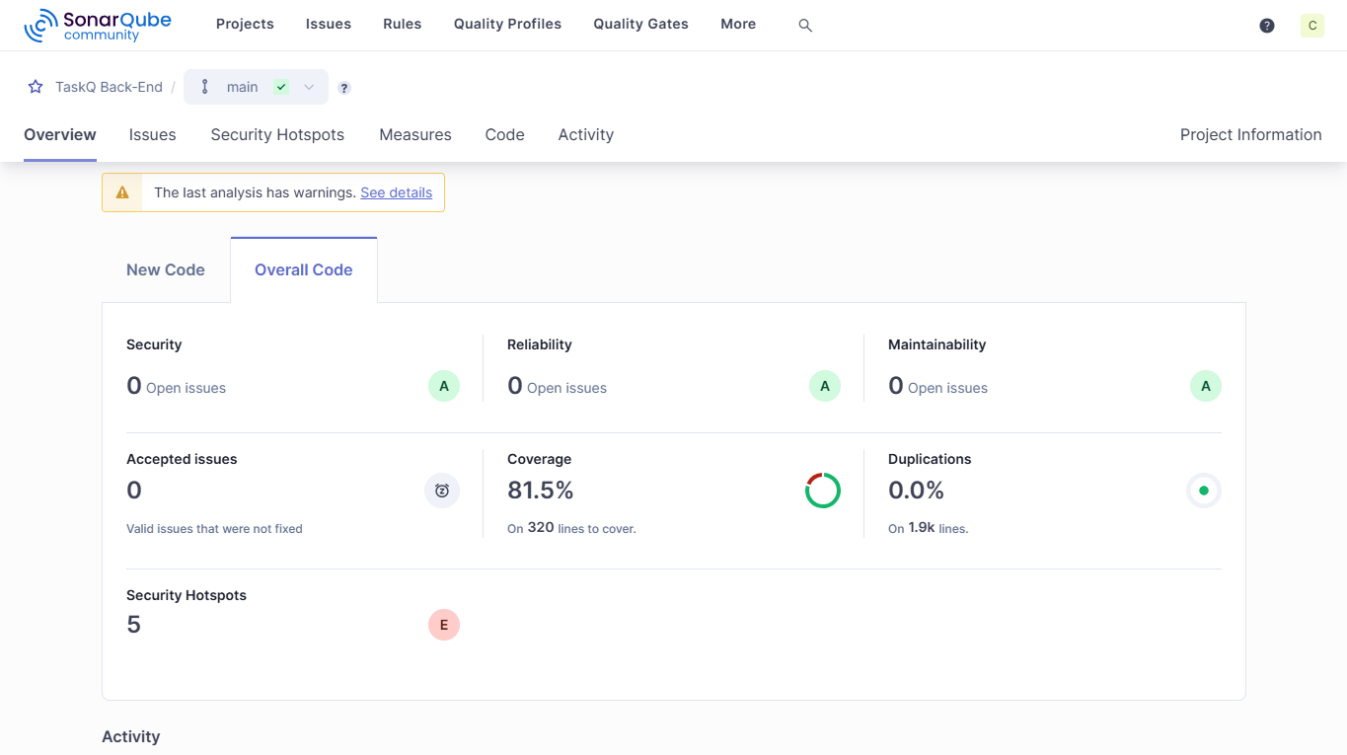
> Decisao: o valor atual nao foi trocado por uma URL inventada. A origem permitida deve ser definida quando a URL do front-end estiver confirmada. Isso evita quebrar a integracao e mantem a correcao limitada ao que foi realmente apontado e verificavel.

5. Conclusao

A segunda analise demonstra evolucao objetiva: todos os 43 code smells foram eliminados e a divida tecnica caiu para zero, sem regressao nos testes, nos indicadores de bugs, vulnerabilidades ou duplicacao. A cobertura subiu para 81,5%. Para chegar a 90%, ainda sera necessario adicionar testes para os fluxos que permanecem sem cobertura; essa etapa nao foi incluida nesta correcao para manter o escopo simples e sem MockMVC.

Evidencia visual - segunda analise do SonarQube

Captura real do painel local em navegador, no modo Overall Code, apos o processamento da segunda analise. O painel mostra zero issues de seguranca, confiabilidade e manutencao, cobertura de 81,5%, duplicacao de 0,0% e cinco hotspots de seguranca.



Quality Gate: Passed | Cobertura: 81,5% | Issues abertas: 0 | Duplicacao: 0,0% | Hotspots: 5